Research Notebook - M2

Topic: Aperiodic Server Algorithms — Family Tree (Genealogy for Priority Exchange)

Note before the table

The Deferrable Server (DS) was originally proposed by Lehoczky, Sha, and Strosnider at the IEEE Real-Time Systems Symposium 1987, but the full IEEE Transactions journal paper appeared later under the author order Strosnider, Lehoczky, Sha (1995) in IEEE Trans. Computers. The community usually cites the 1987 conference paper for primacy.

Priority Exchange was first proposed at the 1988 IEEE Real-Time Systems Symposium as the "Extended Priority Exchange Algorithm" by Sprunt, Lehoczky, and Sha. The 1989 Journal of Real-Time Systems paper (Sprunt/Sha/Lehoczky) is the canonical reference because it consolidates PE and introduces the Sporadic Server in the same paper. Most courses cite the 1989 journal version.

Structured notes — Family tree of aperiodic servers

#	Algorithm	Year	Author(s)	Scheduler	Improves over predecessor	Key mechanism (one sentence)
1	Liu & Layland (foundation)	1973	C. L. Liu, J. W. Layland	RM / EDF	— (no predecessor; defines the field)	Rate Monotonic priority assignment for periodic tasks, with utilization bound $U \leq n(2^{(1/n)} - 1)$.
2	Background Scheduling	pre-1980s (folklore)	— (community origin)	RM (low-priority slot)	— (naive baseline)	Aperiodic tasks run only when no periodic task is ready; poor response time.
3	Polling Server (PS)	≈1985 (analyzed in [1])	Community origin; analyzed in Sprunt/Sha/Lehoczky 1989	RM	Background → bounded response, but capacity is wasted if no aperiodic pending.	Periodic server (T_s, C_s) at high priority; if no aperiodic pending at activation, capacity is DISCARDED.
4	Deferrable Server (DS)	1987 (conf.) 1995 (journal)	Lehoczky, Sha, Strosnider [2]	RM	PS → DS preserves unused capacity until end of period (instead of discarding).	Server (T_s, C_s) keeps its budget until the period ends; aperiodic arrivals during the period are served immediately.
5	Priority Exchange (PE)	1988 (conf.)	Sprunt, Sha, Lehoczky [1, 3]	RM	DS → PE preserves capacity AND maintains the full RM	When no aperiodic is pending, server exchanges priority with the highest-priority ready periodic task; capacity drifts to lower priority levels.

#	Algorithm	Year	Author(s)	Scheduler	Improves over predecessor	Key mechanism (one sentence)
		1989 (journal)			schedulability bound (DS reduces it due to back-to-back effect).	
6	Sporadic Server (SS)	1989	Sprunt, Sha, Lehoczky [1]	RM	PE → SS keeps PE's schedulability bound but uses per-usage replenishment instead of priority exchange — much simpler to implement.	Capacity is replenished only when consumed (after a delay); server can be treated as an ordinary periodic task for schedulability analysis.
7	Dynamic Sporadic Server (DSS)	1994 (conf.) 1996 (journal)	Spuri, Buttazzo [4, 5]	EDF	SS → DSS ports the Sporadic Server idea from RM to EDF.	Same per-usage replenishment as SS, but uses deadline-based priorities; server gets a dynamic deadline.
8	Total Bandwidth Server (TBS)	1994 (conf.) 1996 (journal)	Spuri, Buttazzo [4, 5]	EDF	DSS → TBS achieves FULL processor utilization and shorter aperiodic response, by assigning aperiodic tasks an EDF deadline based on bandwidth allocation.	Each aperiodic request gets a deadline d_k = max(r_k, d_(k-1)) + C_k/Us; aperiodic served by EDF among periodic tasks.
9	Constant Bandwidth Server (CBS)	1998 (RTSS)	Abeni, Buttazzo [6]	EDF	TBS → CBS isolates tasks via temporal protection: a misbehaving task cannot exceed its reserved bandwidth Us.	When a task's budget is exhausted, its deadline is postponed by Ts instead of being suspended; foundation of Linux SCHED_DEADLINE.

Positioning Priority Exchange

- PE was the first algorithm to preserve capacity AND maintain the full RM schedulability bound.
- Trade-off PE made: complexity. The same authors (Sprunt/Sha/Lehoczky) introduced Sporadic Server in the same 1989 paper as a simpler alternative.
- PE is more of a conceptual milestone than a practical algorithm; its successor SS dominates in production RTOS.
- PE has no direct EDF analog. The closest EDF cousin is the Dynamic Priority Exchange (DPE) server by Spuri & Buttazzo (1994), largely subsumed by TBS and CBS.

Open questions for the "sparse related work" section

- PE has very few direct successors — most modern work either implements SS (in RM systems) or moves to EDF-based servers (CBS).
- There is no significant multi-core extension of PE. Modern multi-core RT work uses partitioned scheduling or global EDF with CBS-style reservations.

- The "Extended Priority Exchange" (Sprunt/Lehoczky/Sha 1988) was the only direct extension; even this is rarely cited today.

References

[1]	B. Sprunt, L. Sha, and J. Lehoczky, "Aperiodic Task Scheduling for Hard Real-Time Systems," Journal of Real-Time Systems, vol. 1, no. 1, pp. 27–60, 1989. DOI: 10.1007/BF02341920 Free PDF (CMU SEI tech report version): https://www.sei.cmu.edu/documents/973/1989_005_001_15749.pdf
[2]	J. K. Strosnider, J. P. Lehoczky, and L. Sha, "The Deferrable Server Algorithm for Enhanced Aperiodic Responsiveness in Hard Real-Time Environments," IEEE Transactions on Computers, vol. 44, no. 1, pp. 73–91, Jan. 1995. DOI: 10.1109/12.368008 Original conference version: Lehoczky, Sha, Strosnider, "Enhanced Aperiodic Responsiveness in Hard Real-Time Environments," Proc. IEEE Real-Time Systems Symposium, pp. 261–270, Dec. 1987.
[3]	B. Sprunt, J. P. Lehoczky, and L. Sha, "Exploiting Unused Periodic Time for Aperiodic Service Using the Extended Priority Exchange Algorithm," Proc. 9th IEEE Real-Time Systems Symposium, pp. 251–258, Dec. 1988. DOI: 10.1109/REAL.1988.51120
[4]	M. Spuri and G. Buttazzo, "Efficient Aperiodic Service under Earliest Deadline Scheduling," Proc. IEEE Real-Time Systems Symposium, San Juan, Puerto Rico, pp. 2–11, Dec. 1994. Free PDF: http://retis.santannapisa.it/~giorgio/paps/1994/rtss94.pdf
[5]	M. Spuri and G. C. Buttazzo, "Scheduling Aperiodic Tasks in Dynamic Priority Systems," Journal of Real-Time Systems, vol. 10, no. 2, pp. 179–210, 1996. DOI: 10.1007/BF00360340
[6]	L. Abeni and G. Buttazzo, "Integrating Multimedia Applications in Hard Real-Time Systems," Proc. 19th IEEE Real-Time Systems Symposium (RTSS '98), Madrid, Spain, pp. 4–13, Dec. 1998. DOI: 10.1109/REAL.1998.739726
[7]	C. L. Liu and J. W. Layland, "Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment," Journal of the ACM, vol. 20, no. 1, pp. 46–61, Jan. 1973. DOI: 10.1145/321738.321743
[8]	G. C. Buttazzo, Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications, 3rd ed. New York: Springer, 2011. (Course textbook — covers all the above in Chapter 5)